Applied Intelligence (2026) 56:57
https://doi.org/10.1007/s10489-025-07068-y

A novel membrane-inspired evolutionary algorithm framework for VRPTW

Zhonghai Bai[1] · Václav Snášel[1,2] · Seyedali Mirjalili[1,3,5] · Bay Vo[4] · Lingping Kong[1] · Xiaopeng Wang[1]

Received: 11 April 2025 / Accepted: 16 December 2025
© The Author(s) 2026

Abstract
The vehicle routing problem with time windows (VRPTW) has gained much attention recently due to its wide application in operations research and logistics. VRPTW has been proven to be an NP-hard problem whose optimal solution is computationally costly. Scholars have proposed many methods, such as exact algorithms, heuristics, and metaheuristics, to find near-optimal solutions for the VRPTW. Exact algorithms are limited to small-scale problems, while heuristic algorithms and metaheuristics often converge to locally optimal solutions, despite their applicability to larger-scale problems. This paper proposes a novel membrane-inspired evolutionary algorithm framework (MEAF) consisting of isolated evolutionary rules, communication output rules, communication input rules, fusion-exchange information operation, and membrane dissolution rules. By leveraging the advantages of multiple metaheuristics algorithms and avoiding the pitfalls of local optima, MEAF offers a promising solution to address complex problems. The effectiveness of the proposed MEAF is verified by applying three classical metaheuristics, namely Genetic Algorithm (GA), Ant Colony System (ACS), and Particle Swarm Algorithm (PSO), to solve the VRPTW problem. The experiments are run on 56 instances of Solomon with 100 client benchmarks. The evaluation of the experimental results combined with the mean and standard deviation values show that the algorithm performs better in 54 out of 56 instances, demonstrating the effectiveness and stability of the proposed algorithm.

Keywords Membrane computing · VRPTW · Evolutionary algorithm · Combinatorial optimization · Algorithm optimization

1 Introduction

The Vehicle Routing Problem (VRP), first introduced by Dantzig and Ramser in 1959 [1], is a cornerstone in Operations Research and Logistics. It involves optimizing the fleet organization to serve a group of customers or locations efficiently, with applications in various fields such as cargo delivery, waste collection, public transportation, emergency services, and fleet management. Solving the VRP efficiently

✉ Václav Snášel
vaclav.snasel@vsb.cz

Zhonghai Bai
zhonghai.bai.st@vsb.cz

Seyedali Mirjalili
ali.mirjalili@torrens.edu.au

Bay Vo
vd.bay@hutech.edu.vn

Lingping Kong
lingping.kong@vsb.cz

Xiaopeng Wang
xiaopeng.wang.st@vsb.cz

1 Faculty of Electrical Engineering and Computer Science, VŠB-Technical University of Ostrava, Ostrava 70800, Czech Republic

2 Center for Advanced Technologies and Engineering, CATEN, Ostrava, Czech Republic

3 Centre for Artifcial Intelligence Research and Optimisation, Torrens University Australia, Brisbane, Australia

4 Faculty of Information Technology, HUTECH University, Ho Chi Minh, Vietnam

5 University Research and Innovation Center, Obuda University, 1034 Budapest, Hungary

Published online: 24 January 2026

can lead to significant cost savings, reduced environmental impact, and improved service quality.

Over the years, numerous variants of the VRP have been developed to address specific constraints and requirements, including the Capacitated Vehicle Routing Problem, the Vehicle Routing Problem with Time Windows (VRPTW), the Vehicle Routing Problem with Pickup and Delivery, the Multi-Depot Vehicle Routing Problem, and the Stochastic Vehicle Routing Problem. Among these, the VRPTW is particularly challenging, as it requires not only managing vehicle capacities but also adhering to strict service time windows for each customer. Recent research in this area includes works by Alipour et al. [2], Lehmann et al. [3], Resmi et al. [4], Yue et al. [5], and Zhang et al. [6].

The VRPTW poses significant challenges due to its combinatorial nature, where the number of potential solutions grows exponentially with the problem size. This complexity makes it difficult to find optimal solutions within a reasonable timeframe, especially for large-scale instances. The challenge is further compounded by the need to balance multiple constraints, such as vehicle capacity and customer time windows, which adds layers of complexity to the problem.

Given these challenges, traditional approaches to solving VRPTW can be broadly categorized into exact and heuristic methods [7]. Exact methods, such as branch-and-bound and branch-and-cut, guarantee optimality but suffer from computational infeasibility for larger instances due to the exponential growth of the solution space. Heuristic and metaheuristic algorithms, on the other hand, offer practical and efficient means to obtain high-quality solutions for large-scale problems. These methods include construction heuristics, improvement heuristics, hybrid heuristics, and meta-heuristics. Due to its broad applicability, meta-heuristics represent the most active area in this field. In addition to classical algorithms, such as Genetic Algorithms [8], Simulated Annealing [9], Tabu Search [10], Particle Swarm Optimization [11], and Ant Colony Optimization [12], numerous new algorithms have been proposed recently, such as those by Shehadeh et al. [13], Bouaouda et al. [14], and Dehghani et al. [15].

Despite extensive research on solution methodologies for the VRPTW, a notable gap remains in the practical scalability and adaptability of metaheuristic algorithms, particularly for large-scale instances. While individual metaheuristics such as Genetic Algorithms and Tabu Search have shown promise, their performance often deteriorates when confronted with real-world scenarios involving complex constraints and large problem sizes. Moreover, the absence of a unified and flexible framework for integrating multiple metaheuristics limits the exploitation of their complementary strengths. Existing hybrid or ensemble approaches typically combine algorithms at the operator level or in sequential phases, which can compromise both independence and scalability. Therefore, there is a need for a cooperative framework that allows multiple algorithms to evolve autonomously while exchanging information in a structured and adaptive manner.

Membrane computing provides a natural theoretical foundation for such a cooperative framework. Inspired by the structure and interactions of biological cells, it offers a systematic paradigm for integrating multiple metaheuristics, enhancing both scalability and performance [16, 17]. By leveraging the hierarchical organization and parallel processing capabilities of biological membranes, membrane computing enables effective exploration and exploitation of the solution space.

Based on membrane computing theory, this paper proposes a novel membrane-inspired evolutionary algorithm framework that combines Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Ant Colony System (ACS) to solve the VRPTW. The main innovations and contributions of this paper are:

- **Novel Framework**: We propose a membrane-inspired evolutionary algorithm framework characterized by isolated evolution, structured communication, fusion-based information exchange, and adaptive membrane dissolution. This design emulates the hierarchical and parallel nature of biological membranes and introduces a novel computational paradigm for solving the VRPTW.
- **Hybrid Metaheuristic Integration**: Unlike traditional approaches that rely on a single metaheuristic, MEAF seamlessly integrates GA, ACS, and PSO within a unified structure, enabling complementary cooperation among heterogeneous search strategies to improve both solution quality and computational efficiency.
- **Scalability and Robustness**: Extensive experiments on 56 VRPTW benchmark instances demonstrate that MEAF consistently outperforms its component algorithms in terms of solution quality and runtime efficiency, particularly on large-scale and complex cases, highlighting its scalability and robustness.

The rest of this paper is organized as follows. In Section 2, we review the existing methods for the VRPTW. Section 3 begins with a description of the definition and mathematical model of VRPTW, followed by an introduction to the membrane computing theory, and concludes with the presentation of ACS, GA, and PSO. In Section 4, we propose the membrane evolutionary algorithm framework (MEAF). Section 5 extends MEAF to introduce the MEAVRPTW.

Section 6 presents an experimental comparison and analysis of the MEAVRPTW algorithm against the algorithms that constitute its elementary components. Finally, Section 7 offers some conclusions.

2 Related work

The VRPTW has been extensively studied in the fields of operations research and logistics due to its crucial role in optimizing transportation and delivery systems. Researchers have developed a variety of methodologies to address this complex problem, which can be broadly categorized into two main approaches: exact solution algorithms and approximate solution algorithms.

2.1 Exact solution algorithms

Exact Solution Algorithms for the VRPTW can be classified into three fundamental approaches: Column Generation, Lagrangian Relaxation, and Branch-and-Bound methods. Each method offers distinct strategies for systematically solving the VRPTW, providing valuable insights into the optimization process.

Column ptgeneration is a technique used to solve large-scale linear programs by iteratively adding variables (columns) that can potentially improve the solution. This method was first applied to the VRPTW by Desrosiers et al. [18]. The approach involves initializing the linear program with a subset of variables and solving it. Suppose the solution can be improved by adding new variables (which are chosen based on their potential to reduce the overall cost). In that case, these variables are incorporated into the model, and the process is repeated. This method has been refined over time, with Desrosiers et al. [19] introducing valid inequalities to enhance the method's effectiveness in solving the VRPTW to optimality.

Lagrangian relaxation-based methods offer effective approaches for solving the Vehicle Routing Problem with Time Windows (VRPTW). These methods, including variable splitting and the K-tree approach, have been proposed to optimize VRPTW solutions. Fisher et al. [20] introduced an algorithm that formulates the VRPTW as a K-tree problem with degree 2K on the depot, aiming to find optimal routes. In this method, certain constraints, such as those ensuring each customer is served exactly once, are relaxed using Lagrange multipliers, adding penalty terms to the objective function. Subsequently, a minimum degree-constrained K-tree problem is solved as a subproblem, with Lagrange multipliers adjusted through the sub-gradient approach.

Branch-and-Bound methods are another key approach for solving the VRPTW. Kohl et al. [21] proposed a Branch-and-Bound method, drawing upon the common principle outlined by Balas et al. [22], to explore the VRPTW problem's solution space systematically. This method involves recursively dividing the solution space into smaller subproblems (branches), guided by the calculation of lower bounds for the objective function. By comparing these lower bounds to known upper bounds, subsets of the solution space are pruned, leading to a more efficient exploration. The effectiveness of this approach hinges on the tightness of the bounds, with tighter bounds enabling the removal of more subsets.

2.2 Approximate solution algorithms

In real-world applications, exact solution algorithms for the VRPTW are often impractical due to their high computational cost and time requirements. As a result, approximate solution algorithms are indispensable for finding efficient and practical solutions within a reasonable timeframe. These algorithms can be broadly classified into heuristic and meta-heuristic approaches. Heuristic algorithms, including constructive and improvement heuristics, offer a structured means of building and refining solutions. On the other hand, Meta-heuristic algorithms provide advanced strategies to explore the solution space more extensively, often combining several heuristic principles to escape local optima and enhance solution quality. Below, we delve into these two categories in detail.

Heuristic algorithms applied to the VRPTW can be broadly classified into two categories: constructive heuristics and improvement heuristics. Constructive heuristics build a solution incrementally by generating a partial solution and iteratively selecting and inserting unserved customers into this partial solution until a complete solution is formed, ensuring that all customers are served within their respective time windows and capacity constraints (see studies such as [23–25]). Improvement heuristics, on the other hand, enhance an existing solution by exploring its neighborhood. These algorithms replace the current solution with a better one by examining some or all neighboring solutions. One of the most commonly used improvement heuristics in routing and scheduling is the r-Opt heuristic [26]. It iteratively refines the initial feasible solution by removing r edges and reconnecting the resulting sub-routes in every possible way to form new routes, ensuring that each new solution satisfies

the time window and capacity constraints. Typically, the value of r is set to 2 (2-Opt) or 3 (3-Opt) to balance computational efficiency and solution quality. The r-Opt heuristic reduces the overall routing cost by continuously selecting feasible reconnections, thus efficiently improving the solution until no further improvements can be found.

Meta-heuristic algorithms have also been widely adopted to solve the VRPTW due to their ability to escape local optima and explore a larger solution space. These traditional approaches, such as Genetic Algorithms [8], Simulated Annealing [9], Tabu Search [10], Particle Swarm Optimization [11], Ant Colony System [27], and Whale Optimization Algorithm [28], have demonstrated effectiveness in finding high-quality solutions for this optimization problem. However, recent advancements have focused on enhancing these traditional meta-heuristic methods.

The Self-competitive Particle Swarm Optimization (ScPSO) algorithm introduced by Wang et al. [29] is one such improvement. It optimizes the learning direction of particles based on their self-competition degree, which is defined as the ratio of a particle's fitness value to the population's average fitness. When a particle's self-competition degree is low, indicating superior performance, it learns from its individual optimal solution to achieve rapid convergence. Conversely, a high self-competition degree, indicating inferior performance, compels the particle to learn from better-performing neighbors. This dynamic adjustment of the learning probability factor in each dimension enhances search ability and population diversity. Additionally, ScPSO incorporates a nonlinear inertia weight adaptive strategy to balance exploration and exploitation. It also utilizes random greedy heuristic selection and variable neighborhood search strategies for initial solution construction and constraint management. This comprehensive approach ensures that particles with poor fitness continuously improve by learning from optimal particles, thereby enhancing the overall performance of the swarm.

Moreover, recent research has explored hybrid and advanced optimization methods. Maroof et al. [30] proposed a hybrid genetic algorithm combining random initialization with the Solomon Insertion heuristic to create a diverse initial population, improving exploration and convergence. Widians et al. [31] developed a hybrid Ant Colony and Grey Wolf Optimization (ACO-GWO) algorithm to balance exploration and exploitation, outperforming individual algorithms on benchmark functions despite higher computational cost. Similarly, an adaptive differential evolution with multiple crossover strategies (ADEMCS) [32] enhances both global and local search performance, and a third-order two-stage numerical scheme with neural network simulations for the SEIR model [33] achieves high accuracy for nonlinear problems. These studies illustrate the trend of integrating hybrid, adaptive, and machine learning strategies to improve solution quality and efficiency.

Furthermore, advancements in meta-heuristic algorithms continue to emerge, introducing new approaches to address the VRPTW. These include the Osprey Optimization Algorithm (OOA) [34], which imitates the hunting behavior of ospreys; the Pelican Optimization Algorithm (POA) [35], inspired by the hunting strategies of pelicans; and the Coati Optimization Algorithm (COA) [15], which simulates the natural behaviors of coatis during hunting and escaping from predators. These innovative methods aim to further enhance solution quality and convergence efficiency, contributing to the ongoing progress in optimizing solutions for the VRPTW.

3 Preliminaries

3.1 Problem definition and modeling

VRPTW is a constrained VRP, where the limitations of vehicle capacity Q and customer acceptance time windows need to be considered when optimizing fleet organization to serve a group of customers. The goods delivered to the customers have multiple weights; for one given customer, the goods cannot be split and must be delivered at once. VRPTW can be described with the help of directed graphs. Assume a set $V = \{v_0, ..., v_N\}$ of vertices, let the vertex v_0 represent the depot, and the other vertices $V' = V \backslash \{v_0\}$ are defined as customers. The set of arcs $E = \{(v_i, v_j) | v_i, v_j \in V, i \neq j\}$ represents the set of alternative paths linking the two vertices v_i and v_j. Customer $v_i \in V'$ with location l_i, has a quantity of $q_i > 0$ to be delivered with a time window $[b_i, e_i]$, b_i is the beginning time of customer i allowed to receive goods, and e_i is the ending time allowed to receive goods. A vehicle must wait for time w_i if its arrival time t_i at customer i is earlier than b_i. The travel cost c_{ij} and the passing time t_{ij} between vertices i and j are generally calculated using the Euclidean distance. A set of homogeneous vehicles K with capacity Q are used to serve the customer set V'.

The VRPTW can then be stated mathematically as [36]:

$$Min \sum_{k \in K} \sum_{i \in V} \sum_{j \in V} c_{ij} x_{ijk} \tag{1}$$

subject to:

$$x_{ijk} = \begin{cases} 1, & \text{if vehicle } k \text{ travels from } v_i \text{ to } v_j \\ 0, & \text{otherwise} \end{cases} \tag{2}$$

$$\sum_{k\in K}\sum_{i\in V, i\neq j} x_{ijk} = 1 \quad (\forall j \in V) \tag{3}$$

$$\sum_{k\in K}\sum_{j\in V, j\neq i} x_{ijk} = 1 \quad (\forall i \in V) \tag{4}$$

$$\sum_{i\in V'} q_i \sum_{j\in V', j\neq i} x_{ijk} \leq Q \quad (\forall k \in K) \tag{5}$$

$$\sum_{i\in V'} x_{i0k} = \sum_{j\in V'} x_{0jk} = 1 \quad (\forall k \in K) \tag{6}$$

$$w_i = max\{b_i - t_i, 0\} \quad (\forall i \in V') \tag{7}$$

$$t_j = t_i + w_i + s_i + t_{ij} \quad (\forall i \in V', i \neq j) \tag{8}$$

$$t_j + t_{j0} \leq e_0 \quad (\forall j \in V') \tag{9}$$

$$b_i \leq t_i + w_i \leq e_i \quad (\forall i \in V') \tag{10}$$

The objective function (1) is to minimize the total travel cost during delivery. Constraints (3) and (4) guarantee that each customer is served precisely once, while (5) states that no vehicle can be loaded beyond its capacity. Equation 6 states that each vehicle leaves depot 0 and eventually returns to depot 0. Equation 7 gives the waiting time of customer *i*, and (8) gives the arrival time from customer *i* to *j*, where s_i is the service time at customer *i*. Equation 9 states that a vehicle must have sufficient time to return to the depot after servicing customer *j*. Finally, constraint (10) indicates that the serving time adheres to the time window.

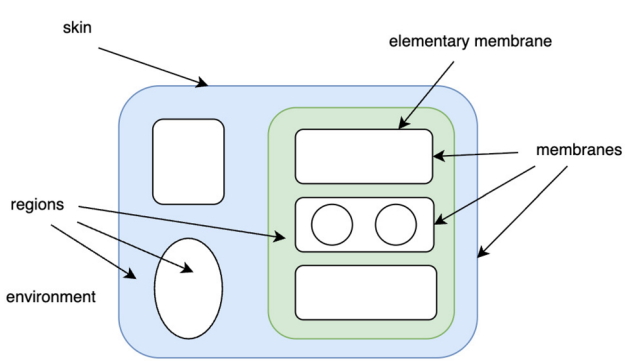

Fig. 1 Illustration of a hierarchical membrane structure. Each membrane represents a distinct computational compartment that evolves its internal objects according to specific rules. The skin membrane encloses the entire system, while inner membranes (elementary or non-elementary) define local regions that may contain other membranes or objects. Communication rules enable the exchange of information between adjacent regions, illustrating how membrane computing models parallel and cooperative computation. As an analogy, one may imagine a population-based optimizer (e.g., a GA) evolving within one membrane and occasionally exchanging solutions with others

3.2 Membrane computing

Membrane computing (also known as membrane systems or P systems), introduced by Păun in 2000 [37], is a computational paradigm inspired by the organization and functioning of biological cells. It models complex systems by mimicking biological processes such as compartmentalization, communication, and evolution.

Intuitively, a membrane system can be viewed as a hierarchy of compartments (membranes), each containing objects and rules that govern their transformation and movement. These membranes can represent independent computational units that interact through well-defined communication mechanisms. For instance, one may imagine a population-based optimizer such as a Genetic Algorithm (GA) evolving within one membrane, which occasionally exchanges information with others through communication rules. This analogy helps illustrate how membrane computing captures the idea of parallel and cooperative computation. Figure 1 shows a simple hierarchical membrane structure.

Formally, a cell-like membrane system is defined as:

$$\Pi = (O, H, \mu, \omega_1, ..., \omega_m, R, i_o), \tag{11}$$

where:

- *O* is an alphabet whose elements are called *objects*;
- *H* is a finite set of labels for membranes;
- μ represents the hierarchical membrane structure consisting of *m* membranes labeled by *H*;
- ω_i $(1 \leq i \leq m)$ are multisets over *O* associated with the regions of μ;
- *R* is a finite set of rules governing evolution, communication, and membrane-handling operations;
- $i_o \in H$ denotes the output membrane.

During computation, each membrane evolves its contained objects according to the rules in *R*. Objects can move between membranes (communication), membranes can merge or dissolve, and the process continues until no rule can be applied, at which point the system halts. A configuration of a P system is thus determined by both the membrane structure and the multisets of objects contained within each compartment. For more detailed discussions on the theoretical foundations of P systems, readers may refer to [38].

In summary, membrane computing provides a natural model for distributed and cooperative computation, where multiple compartments evolve in parallel while exchanging information. This paradigm has inspired a variety of algorithmic frameworks, including our proposed MEAF model,

which applies these concepts to enable cooperative metaheuristic optimization.

3.3 Ant colony system

This part describes the ant colony system (ACS) [27] applied to the traveling salesman problem (TSP). Both TSP and VRTPW are combinatorial optimization problems, along with the ACS, that can be applied to the VRPTW problem. It is an improved algorithm of ant system (AS) [39] in terms of efficiency. Regarding the original ant system, it is a meta-inspired algorithm inspired by the behavior of ants searching for food. A pheromone is a means ants use to communicate during the food search. The more pheromones there are, the more attractive they are to the ants. Assuming that all ants release the same amount of pheromones from the starting point to the destination, ants that choose shorter paths will leave more pheromones per unit length of their travel path. Considering pheromones' effect on ants, all ants will choose the shorter path to reach the food destination after searching. The ACS improves the transition rules and pheromone update rules to make the algorithm more efficient.

The ACS algorithm consists of four key steps, i.e., pheromone initialization, transition rules, local update rules, and global update rules, as shown below.

Pheromone initialization The choice of the initial pheromone level can have an impact on the performance of the algorithm. Setting the initial pheromone level too high can lead to early convergence to a sub-optimal solution, while setting it too low can slow down the search process. It was found that initialized with 12 is a good initial value,

$$\tau_{initial} = (n \cdot L_{nn})^{-1}, \tag{12}$$

where L_{nn} is the tour length produced by the nearest neighbor heuristic [40] and n is the number of cities.

Transition rule In ACS, transition rules consist of exploration and exploitation strategies, using a parameter q_0 to decide which strategy to use.

$$P_k(r,s) = \begin{cases} \frac{[\tau(r,s))]\cdot[\eta(r,s)]^{\beta}}{\sum_{\mu\in J_k(r)}[\tau(r,u))]\cdot[\eta(r,u)]^{\beta}}, & \text{if } s \in J_k r \\ 0, & \text{otherwise} \end{cases} \quad \text{if } q > q_0 (\text{exploration}) \tag{13}$$

$$\underset{\mu\in J_k(r)}{arg\ \ max} \left\{[\tau(r,u))] \cdot [\eta(r,u)]^{\beta}\right\}, \text{ if } q \le q_0 (\text{exploitation}), \tag{14}$$

where q is a random number uniformly distributed in [0, 1], q_0 is a parameter $0 \le q_0 \le 1$, $\tau(r,s)$ means the pheromone value between city r and s, $\eta(r,s)$ is the inverse of the distance between city r and s, $\beta > 0$ is used to determine the relative importance of the pheromone information compared to the distance information, and $J_k r$ is the set of cities not yet visited by the ant k on city r. Each time ant k on city r selects the next city s from the set of unvisited cities $J_k r$ according to these two strategies.

Global update rule In AS, all constructed solutions were used to update the pheromone after one iteration, but in ACS, only the global best Ant was allowed to add a pheromone after each iteration, which means that only those edges belonging to the global best trip were enhanced. With this change, ACS has proven to be more effective than AC.

$$\tau(r,s) \leftarrow (1-\rho)\cdot\tau(r,s) + \rho \cdot \Delta\tau_k(r,s), \tag{15}$$

where

$$\Delta\tau_k(r,s) = \begin{cases} (L_{gb})^{-1}, & \text{if } (r,s) \in \text{global best route} \\ 0, & \text{otherwise} \end{cases}$$

$0 < \rho < 1$ **represents the pheromone evaporation parameter,** L_{gb} **is the length of the global best route searched since the trial.** L_{gb} **could be substituted as** L_{ib}**, called iteration-best, which is the length of the best tour in the current iteration of the trial.**

Local update rule In addition to applying global update rules at the end of each iteration, ACS also uses local update rules for *arc*(i, j) each time an ant migrates from node i to node j during the tour construction.

$$\tau(r,s) \leftarrow (1-\rho)\cdot\tau(r,s) + \rho \cdot \tau_{intial} \tag{16}$$

The local updating rule decreases the concentration of pheromones on an already visited arc, making the arc less desirable for a following ant. This way, the other ants' exploration probability of new paths is increased.
When applied to the VRPTW problem, if the VRPTW constraint is not satisfied when the next location is selected, it may try to execute the transition rule again to regenerate the next location and assign a new vehicle if it is still not satisfied. Equation 16 is only used when it may decrease the pheromone.

3.4 Genetic algorithm

In this part, we briefly review the genetic algorithm. This algorithm is inspired by natural selection, which chooses the most suitable individuals to reproduce the next generation. There are five main steps: population initialization, evaluation, selection, crossover, and mutation.

Step1. Population initialization. One population means one solution to the problem. The population size is a parameter that could be adjusted depending on the nature of the problem. The initial population could be generated randomly, which means solutions are generated from all the search space. Some methods can be used for specific problems to "seed" solutions in regions where the best solution may be available. For example, in the TSP problem, the nearest neighbor heuristic [40] can initialize the path.

Step2. Evaluation. The evaluation procedure needs a fitness function to evaluate the quality of each generated solution. The fitness function varies from problem to problem, e.g., in the Travel Salesman Problem (TSP), the fitness function could be the total distance traveled, while in a classification problem, it may be the correct classification rate. The fitness score of each individual will be used in the selection procedure in the current study.

Step3. Selection. The selection phase is choosing the parents for mating and passing their genes to the next generation. There are mainly three ways to select the parent: Fitness Proportionate Selection, Truncation Selection, and Tournament Selection [41]. However, it also could be a combination of these methods. For instance, the Truncation Selection could be used to choose 20 percent as the candidate parents, then the Fitness Proportionate Selection could be used to select from candidate parents to generate new generations.

Step4. Crossover. Crossover is mixing and matching two selected parents' DNA to produce new offspring. There are various crossover techniques, e.g., one-point crossover, k-point crossover, uniform crossover, etc. [42]. In terms of combinatorial problems, Partially Mapped Crossover (PMX) [43], Order Crossover (OX) [44], and Position-Based Crossover (POS) are often used.

Step5. Mutation. Mutation introduces diversity to the algorithm and gives the algorithm chances to escape from local minima. A mutation rate represents the percentage of the mutation procedure that happens to each offspring. In combinatorial problems, several mutation ways are described in the paper [45], e.g., reverse sequence mutation, the choice of two random positions *i*, *j* in a solution, such that $i < j$, and inverse element between *i* and *j*. The procedure is shown in Fig 2.

When applied to the VRPTW problem, vehicles must be assigned to the distribution sequence under the VRPTW constraint. The task sequence is assigned to a vehicle when the constraint is satisfied, and new vehicles continue to take up the remaining tasks in the sequence when it is not satisfied.

3.5 Particle swarm optimization

3.5.1 Standard PSO algorithm

In this section, the conventional particle swarm optimization (PSO) is introduced briefly. It is a metaheuristic optimization algorithm proposed by Kennedy and Eberhart(1995) [11]. The algorithm is inspired by the social behavior of a flock of birds or a school of fish. The algorithm begins by generating a group of random particles with random positions and velocities. Each particle position represents a potential solution. The solution could be evaluated through a designated fitness function supposing an optimization problem has D variables to be adjusted. By applying the PSO algorithm, the *i*th particle's position could be represented by a D-dimensional vector: $X_i = (x_{i1}, x_{i2}, x_{i3}, x_{i4}, ..., x_{iD})(i = 1, 2, ..., m)$, where *m* represents the swarm size. Furthermore, the *i*th particle's moving velocity could also be denoted as a D-dimensional vector: $V_i = (v_{i1}, v_{i2}, ..., v_{iD})(i = 1, 2, ..., m)$. In PSO, each particle's position in the search space is updated by adjusting its velocity. The velocity update is guided by two factors: the particle's own experience and the experience of the swarm. One particle's own experience refers to the best position found by this particle so far, which is called "*pbest*". The experience of the swarm refers to the best position found by any particle in the swarm so far, which is called the "*gbest*". The updated rules of the position and the velocity of the *i*-th particle are shown as follows:

$$\begin{cases} X_i(k+1) = X_i(k) + V_i(k+1) \\ V_i(k+1) = wV_i(k) + c_1 r_1 (pbest_i - x_i(k)) + c_2 r_2 (gbest - x_i(k)) \end{cases}, \quad (17)$$

where *w* is the inertia weight ranges from 0 to 1; c_1 and c_2 are positive constants, called learning rates, and r_1 and r_2 are random numbers in the interval (0, 1). The best particle is the approximate optimal solution when the maximum number of iterations is reached or attain a satisfactory fitness value. The standard PSO algorithm is suitable for

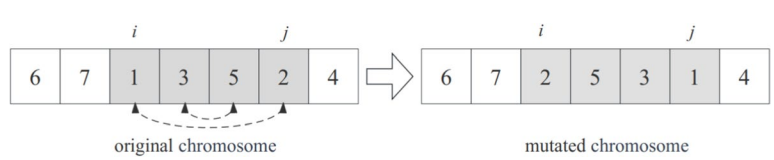

Fig. 2 A diagram of reverse sequence mutation

solving persistent problems with a continuous search space. PSO can also be modified to handle discrete problems.

3.5.2 Discrete PSO algorithm

Since we know that VRPTW is a constrained TSP, we briefly describe applying the discrete PSO algorithm to solve the TSP. Some studies analyzed the details of using it to TSP, e.g., Clerc [46], Hendtlass [47] and Wang [48] proposed different PSO methods for solving TSP problems. Here we introduce the technique proposed by Wang [48].

Swap Operator Assume a solution sequence of TSP as $S = (a_i), i = 1, ..., n$, where n is the city size, and a_i represents the city number. Then, Swap Operator $SO(a_i, a_j)$ is defined as swapping the city number at positions *i*, *j* in solution S.

Velocity An ordered sequence of swap operators is defined as particle velocity, e.g., $SS = (SO_1, SO_2, ..., SO_n)$, where $SO_1, SO_2, ..., SO_n$ are swap operators. The operator "$\oplus$" is used to define one velocity merge with another velocity. Let SS_1 and SS_2 be two Swap Sequences. In TSP context, compared to the direct calculation of SS_1 merge SS_2; usually, SS_1 and SS_2 acted on one solution S in order, namely operating SS_1 first, operating SS_2 second to get the new solution S'. Let $\|SS'\|$ be the length of the directly merged sequence of SS_1 and SS_2, we have $\|SS'\| \leq \|SS_1\| \oplus \|SS_2\|$.

Position plus velocity Applying a velocity $SS = (SO_1, SO_2, SO_3, ..., SO_n)$ on a solution S to obtain a new position S' could be described as $S' = S + SS$, which means that all the swap operators of the SS act on the solution in order. We denote the plus sign "+" as Swap Operators on the solution.

Position minus position Let A and B be two solutions. Assuming a swap sequence S' applying it to B gives A, the sequence S' was defined as the result of $A - B$. Here the minus sign "-" has a different meaning. Using the signs introduced to describe the definition, we get $B + S' = A => S' = A - B$. Equation 17 can now be rewritten as follows:

$$\begin{cases} X_i(k+1) = X_i(k) + V_i(k+1) \\ V_i(k+1) = wV_i(k) \oplus r_1(pbest_i - x_i(k)) \oplus r_2(gbest - x_i(k)) \end{cases}, \tag{18}$$

where w, r_1, r_2 have the same meaning and ranging scale as they were in (17). r_1 multiply $(pbest_i - x_i(k))$ means that all Swap Operators contains in Swap Sequence $(pbest_i - x_i(k))$ will be retained by probability r_1, which is the same as $r_2(gbest - x_i(k))$ and $wV_i(k)$. Each particle represents a permutation of cities, and the fitness function is the total distance of the tour. Substitute the concept introduced above with the standard PSO; then, it's the discrete PSO for TSP. The remaining processing is the same as the GA algorithm when applied to the VRPTW problem.

Metaheuristic algorithms possess advantages such as flexibility, global optimization capability, computational efficiency, and ease of implementation. However, they also have drawbacks, including the tendency to converge to local optimal solutions, sensitivity to parameter settings, and the need for different algorithms to adapt to specific problem characteristics. To maximize the benefits of metaheuristics while mitigating their drawbacks, our proposed framework focuses on fostering the cooperative evolution of multiple metaheuristics. Drawing on the principles of the membrane system, the framework enables the independent operation of diverse evolutionary algorithms within membranes governed by rules that facilitate effective communication and the combination of their evolutionary strategies. By doing so, we address issues like parameter sensitivity and the tendency to converge on local optimal solutions, which are limitations typically associated with individual metaheuristics.

4 A new membrane evolutionary algorithm framework

4.1 Membrane structure for the framework

In this subsection, the one-level membrane structure used in the framework is shown in Fig 3. The structure consists of a skin membrane labeled with 0 representing the overall algorithmic operating environment and *n* elementary membranes labeled from 1 to *n* representing the operating environments of different evolutionary algorithms. The elementary membrane can release the current optimal candidate solution set top-k $Best_i$ generated by algorithm *i* into the skin region or obtain the optimal candidate solutions released by other algorithms from the skin region. Furthermore, the elementary membrane can be dissolved according to the dissolution rules, and its populations can be rearranged into other membranes or disappear.

4.2 Proposed framework

In this subsection, we describe the details of the MEAF based on the proposed membrane structure, which explores the problem space by superimposing the advantages of multiple evolutionary algorithms to obtain an approximate optimal solution.

Alg 1 describes the process in detail. During the initialization procedure, the membrane structure is created. In this

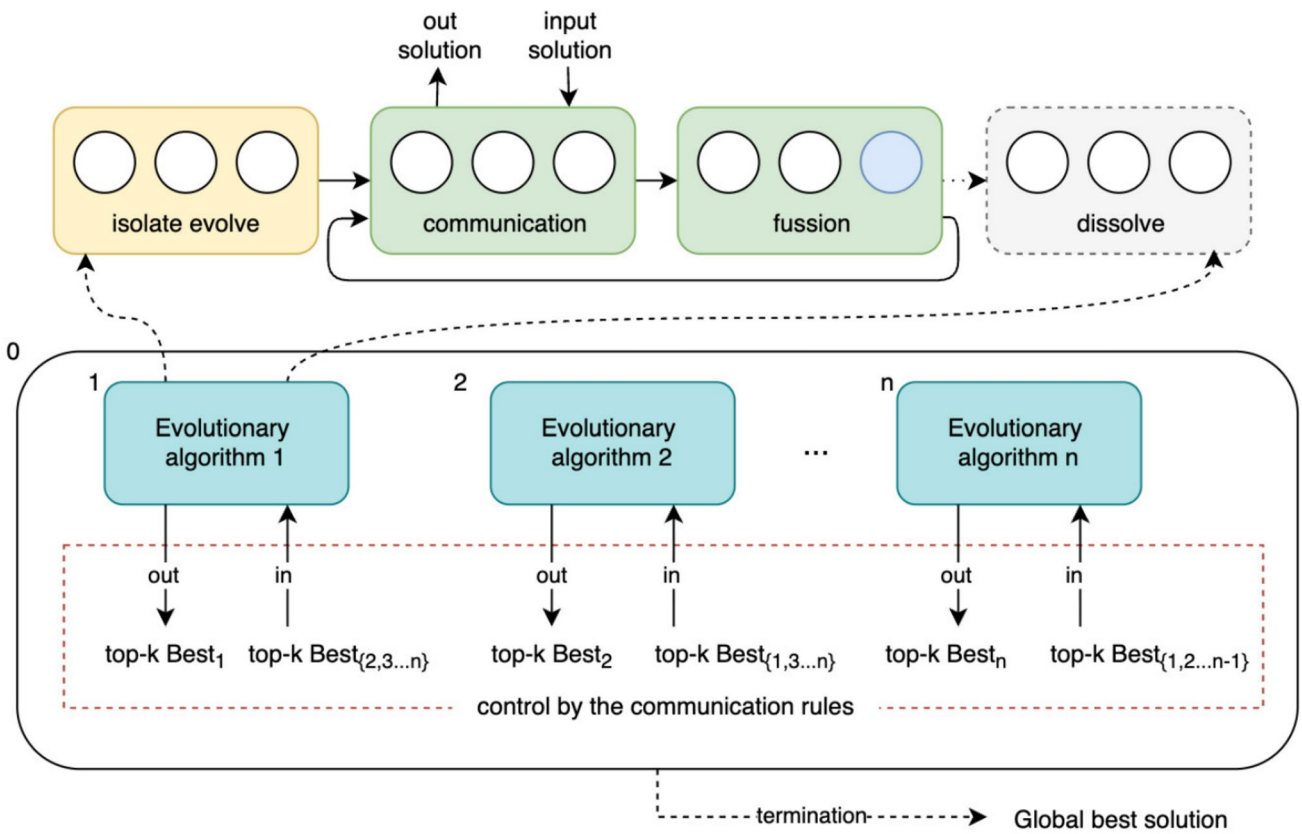

Fig. 3 Membrane structure proposed in the MEAF framework. Each metaheuristic evolves independently in its own membrane (Isolate Evolution). Membranes periodically exchange information (Communication), and the Fusion phase integrates the best external solutions to guide further evolution (e.g., the GA membrane can directly use external best solutions as candidate parents). Underperforming membranes may be removed (Dissolve). The framework outputs the final global solution, representing the best solution across all algorithms. The inset illustrates key operations and interactions among membranes

structure, each elementary membrane hosts a population for a specific evolutionary algorithm (EA). The populations are initialized independently within their respective membranes, allowing for diverse initialization methods tailored to each EA. For the VRPTW, each individual in the population represents a solution—a set of transport routes that meet the constraints.

The primary process can be divided into two phases: isolated evolution and cooperative evolution.

Isolated evolution phase It involves each elementary evolutionary algorithm (EA) operating independently within its membrane, carrying out evolutionary steps like selection, crossover, mutation, and fitness evaluation without interacting with other EAs. This isolation enables each algorithm to leverage its unique strengths and strategies to explore the solution space effectively. By allowing multiple approaches to independently explore different regions of the solution space, this phase enhances diversity and helps avoid premature convergence on suboptimal solutions, which is a common risk when relying on a single algorithm.

Cooperative evolution phase The EAs begin to interact once they reach a certain state, such as completing a predefined number of generations or meeting a convergence criterion. During this phase, information exchange between the different EAs occurs, which may include sharing the best solutions or exchanging genetic material. Each EA continues its evolutionary process independently but now incorporates the shared information. This hybrid approach aims to refine and enhance solutions through collaboration, leveraging the collective strengths of all participating algorithms while maintaining their individual identities and methodologies.

Different rules govern the transitions and interactions between these phases. For instance, the frequency and type of information exchange during the cooperative evolution

phase are crucial parameters that influence the effectiveness of the MEAF. These rules are designed to balance exploration and exploitation across the combined search space, ensuring a robust search process.

The dual-phase structure of isolated and cooperative evolution is a key feature of the MEAF. In the isolated phase, the emphasis is on leveraging the specific advantages of individual EAs to explore different solution strategies. During the cooperative phase, the focus shifts to integrating these diverse strategies to enhance overall solution quality. This approach not only helps in achieving a more comprehensive search, but it also facilitates convergence towards high-quality solutions.

By superimposing the strengths of multiple evolutionary algorithms, MEAF provides a flexible and powerful framework for solving complex optimization problems like VRPTW. The structured yet dynamic interaction between isolated and cooperative evolution phases enables MEAF to adaptively balance exploration and exploitation, leading to the discovery of high-quality solutions that might be challenging to achieve with a single algorithm alone.

Algorithm 1 Membrane Evolution Algorithm Framework.

```
Input: initialization parameters for each elementary
       evolutionary algorithm(EA), max iterations, isolate
       evolutionary rule, communication out rule,
       communication in rule, and dissolve rule.
Output: route solution
1)Initialization
  initial population of each EA
2)isolated evolve
  foreach EA do
    while isolate evolutionary rule is satisfied do
      operate EA for one iteration
      iteration + 1
    end
  end
3)Cooperated evolve
  repeat
    foreach EA do
      if dissolve rules is satisfied then
        Dissolve this EA
      end
      else
        if communication in rules is satisfied then
          update local exchange info
        end
        if communication out rules is satisfied then
          emission top-k solution to the outside
          membrane
        end
        fusion exchange information and evolve
      end
      iteration + 1
    end
  until iteration<max iterations;
```

4.3 Evolve operators

This subsection explains the five key operators inspired by membrane systems (isolated evolutionary rules, communication out rules, communication in rules, fusion exchange information, and membrane dissolve rules) contained in Alg 1. These operators enhance the framework's flexibility and efficiency, facilitating adaptation to diverse algorithm combinations.

1) Isolated evolutionary rules

After the initialization process, the candidate solutions within each base membrane are the original solutions before they evolved. Population exchange at this point will only result in meaningless communication loss. Population communication among each membrane must be performed after their fundamental evolution. There can be several ways to evaluate whether the fundamental evolution is complete. One is the number of independent evolution iterations, and the other is the number of iterations where the optimal solution does not change. We named this type of rule, an isolate evolution stable rule.

2) Communication out rules

After satisfying the run to the isolate evolution stable rule, each algorithm outputs its current top k optimal solution to the skin region, where $k \geq 1$. The time of the following output to the outside can be determined by the number of run intervals or by whether more optimal solutions are produced. Algorithm dimension, the latest outcomes replace the previous ones.

3) Communication in rules

The rule of communication is usually associated with the rule of communication output, either operating an input when a better solution is available externally or after a fixed iteration. However, the communication in rules can also differ for different elementary membranes.

4) Fusion exchange information

There are various strategies for incorporating the exchanged information into specific evolutionary algorithms, which can be as simple as treating the exchanged populations directly as ordinary individuals joining the cluster to participate in the subsequent evolutionary process. However, this is a simple and general way of fusion, which may suffer from possible inefficiencies. A better approach is to develop different strategies depending on the specific evolutionary algorithm. In the next section, we will subsequently describe the fusion strategies of ACS, PSO, and GA.

5) Membrane dissolve rules

In the proposed MEAF structure, each basic EA occupies a computational power, which is an expensive resource; thus, some inefficient algorithms must be eliminated from the run time. Therefore, dissolution rules are used here. The algorithm execution process identifies the algorithms that

cannot produce quality solutions. One of the identification strategies is described below:

Algorithm 2 Dissolve rule.

Input: each EA's so far best solution(solution set), iteration, $df_{percent}$, sd_{iter}, $eas_{threshold}$
Output: one EA or $\emptyset$

```
begin
    if iteration < sd_iter then
        return ∅
    end
    if size of solution set ≤ eas_threshold then
        return ∅
    end
    best fitness=min(solution set);
    worst fitness=max(solution set);
    if best fitness/worst fitness < df_percent then
        return EA has the worst fitness
    end
    else
        return ∅
    end
end
```

A specific dissolution identification rule is given in Alg 2, where it is assumed that the goal is to minimize the value produced by the fitness function. The parameters sd_{iter} and $eas_{threshold}$ denote the number of earliest starting dissolution iterations and the number of minimum evolutionary algorithm instances, respectively. The dissolve rule first determines the number of iterations of the algorithm and then the number of surviving algorithms. When both constraints are satisfied simultaneously, the evaluation of the results produced by the algorithm is started. Finally, the decision to dissolve the algorithm is made by determining whether the ratio of the optimal solution to the worst solution is less than $df_{percent}$, where $df_{percent}$ is a positive constant less than 1.

MEAF operates as a cooperative hybrid framework integrating ACS, PSO, and GA. Each algorithm evolves independently at the population level, maintaining its own search strategy and internal logic, analogous to developing its own "culture." Periodically, the best solutions are shared among algorithms, allowing them to learn from each other without interfering with their self-contained evolution. Unlike conventional hybrid or ensemble metaheuristics that combine algorithms through operator-level mixing or sequential coordination, MEAF preserves full algorithmic autonomy while enabling membrane-inspired, structured communication among them. This collaborative exchange leverages complementary strengths, accelerates convergence, and preserves population diversity, offering advantages over both single-algorithm and traditional hybrid approaches. The key parameters of the MEAF framework are summarized in Table 1.

5 MEAVRPTW

This section introduces a specific application of the MEAF framework to solve the VRPTW. In selecting ACS, GA, and PSO as the elementary evolutionary algorithms, we have carefully considered their foundational significance and diverse strengths.

ACS and GA are recognized for their robust convergence capabilities in discrete optimization problems like VRPTW. Genetic Algorithms (GAs) utilize selection, crossover, and mutation to explore large search spaces effectively. Ant Colony System (ACS) excels in dynamic and combinatorial tasks through pheromone trails and probabilistic solution construction. PSO, typically used for continuous optimization, may not perform optimally in a discrete form. However, including PSO broadens the scope of optimization strategies and tests the MEAF framework's ability to exclude unsuitable algorithms. PSO's unique concepts like velocity and position updates contribute to its strengths in continuous spaces. This allows us to assess the framework's adaptability. Many newer meta-heuristic algorithms lack significant novelty and often repurpose existing methods without substantial improvements. By contrast, ACS, GA, and PSO have demonstrated their foundational impact and broad applicability across various problems.

In summary, ACS, GA, and PSO were selected for their diverse strengths, foundational roles, and to test the framework's ability to exclude poorly performing algorithms. This strategic choice enables a comprehensive exploration of optimization strategies and underscores the MEAF

Table 1 MEAF Framework: Parameter Definitions and Defaults

Parameter	Explanation	Default/Range
T	Minimum iterations to keep the same solutions before exchange (isolated evolution)	5
M	Iterations between successive exchanges (communication interval)	15
sd_{iter}	Earliest iteration to start applying dissolve rules	50
$eas_{threshold}$	Minimum number of EA instances required to avoid dissolve	2
$df_{percent}$	Dissolve factor threshold (best/worst fitness ratio)	0.85
k	Number of top solutions exchanged between membranes	1
fusion	How exchanged solutions are incorporated into each EA	dependent on EA type
max_{iter}	Maximum number of iterations for framework convergence	according to experiment

framework's adaptability. The subsequent parts will detail the integration of these algorithms and the computational complexity of the MEAVRPTW.

5.1 Evolve rules specification

In this subsection, we specify the rules needed in the framework.

Isolate evolution stable rule We use (19) as shown below:

$$\begin{cases} optSol_i = optSol_j \\ j - i \geq T_{iter}, \end{cases} , \tag{19}$$

where $optSol_i$ means the optimal solution until iteration *i*, T is a positive integer. The (19) implies that the optimal solution hasn't changed in T_{iter} iterations.

Communication in and out rule We combine them and make inputs whenever new outputs are available. This is described in the following equation:

$$iter_{now} - iter_{out} \geq M \ , \tag{20}$$

where $iter_{now}$ means the iteration times until now, $iter_{out}$ means the iteration times of the last communication out, and M is a positive integer. The Eq 20 implies that the communication happens every M iteration.

Dissolve rule Here, we use the strategy mentioned in Alg 2. When the selected algorithm is eliminated, the individuals within it are directly eliminated along with the algorithm.

5.2 Fusion strategy specification

In terms of evolving with the exchanged information, we will introduce the method used on each algorithm we selected.

1)ACS As shown in Section 3.3, the core operations of the ACS are on pheromones, including local pheromone updates and global pheromone updates. Here we treat externally introduced external solutions as global optimal solutions. The top-k method filters the externally oriented solutions, i.e., the best k of the externally introduced solutions is used for pheromone updates. Exceptionally, when k equals the number of external solutions, the strategy degenerates to apply a single (15) to each externally optimal solution introduced by the exchange to update its pheromone.

2)PSO The core of the particle algorithm is to update the velocity of the particles, as shown in (17). For an externally introduced optimal solution, we can treat it as a local optimal solution and a global optimal solution. Equation 17 can be turned into the following form:

$$\begin{cases} X_i(k+1) = X_i(k) + V_i(k+1) \\ V_i(k+1) = wV_i(k) \oplus r_1(P_i - x_i(k)) \oplus r_2(P_g - x_i(k)) \ldots \\ \oplus r_3(P_{o1} - x_i(k)) \oplus \ldots \oplus r_{n+2}(P_{on} - x_i(k)) \end{cases} , \tag{21}$$

where $r_1, \ldots, r_{n+2}$ are random numbers in the interval (0, 1), $P_{o1}, \ldots, P_{on}$ are exchanged optimal solutions, and *n* is the size of exchange optimal solutions.

3)GA In the selection step of the genetic algorithm, we first select candidate parents from the population using truncated selection. Next, we add external exchange solutions to the candidate parents. Finally, we use the fitness proportionate selection method to select the parents to generate new offspring.

5.3 Computational complexity

The computational complexity of MEAVRPTW depends on the complexity of the basic algorithms that compose it. Here, we first need to present the computational complexity of the three basic algorithms. In VRPTW, the computational complexity of the GA algorithm is referred to in article [49] as $O(I \times N \times D)$, the computational complexity of the PSO algorithm is referred to in article [50] as $O(I \times N \times D)$, and the computational complexity of the ACS algorithm is is referred to in article [51] as $O(I \times N \times D^2)$, where *I* denotes the number of iterations, *D* denotes the dimensionality of the problem (i.e., the number of clients), and *N* denotes the number of populations. The MEAVRPTW algorithm is a combination of the above three algorithms. The percentage of each basic algorithm is affected by the parameters, which are $O(I \times N \times D^2)$ in the worst case, and $\Omega(I \times N \times D)$ in the best case.

6 Experiments and results analysis

In the experiments, we chose Solomon's VRPTW benchmark problem with 100 customers [52]. The benchmark problem consists of six sets R1, R2, C1, C2, RC1, and RC2. The problem sets C1 and C2 are generated by clustering, R1 and R2 are randomly generated, and RC1 and RC2 are a mixture of random and clustering. The property of vehicle capacity is small in problem sets R1, C1, and RC1. In contrast, the vehicle capacity is large in problem sets R1, C1, and RC1. The objective function is to minimize the total transport distance of the delivery solution.

This section is organized as follows:

- In the **Experiment setup**, we describe the parameters and experimental environment used for evaluating the algorithms, including an introduction to the strategy for adjusting parameters.
- In the **Result analysis**, we first present and analyze the performance of our proposed MEAVRPTW algorithm compared to elementary algorithms (ACS, PSO, and GA). Then, we introduce a comparison with recently published algorithms such as Coati Optimization Algorithm (COA) [15], Osprey Optimization Algorithm (OOA)[34], and Pelican Optimization Algorithm (POA) [35].

6.1 Experiment setup

The algorithm proposed in this article is implemented in Java programming language. All experiments were conducted on macOS based on 4 Intel i7 2.2 GHz CPU cores and 16GB RAM.

The parameter settings for the algorithms used in this paper are detailed in Table 2. These settings were chosen based on a strategy that combines empirical knowledge, literature review, and preliminary experiments. This part introduces the strategy for tuning parameters within our MEAF framework, focusing on framework-specific parameters. The parameters for GA, PSO, and ACS follow their original algorithms, with well-documented tuning strategies in existing research.

Table 2 Parameter settings for the algorithms used in this paper

algorithm name	parameter explanation	value
Ant Colony System	colony size	100
	parameter ρ for pheromone evaporation	0.3
	parameter q_0 for exploitation or exploration control	0.1
	parameter β for distance influence control	2
	parameter top-k for filter the exchange solution	1
Genetic Algorithm	population size	100
	parameter *ts* for truncation selection	0.2
	parameter *mr* for mutation ration	0.5
Particle Swarm Optimization	swarm size	100
	parameter *w* for inertia weight	0.1
MEAVRPTW	parameter *T* for at least keep the same time before exchange	5
	parameter *M* for iteration of exchange intervals	15
	parameter $eas_{threshold}$ for the minimum number of EA instances	2
	parameter sd_{iter} for the earliest start dissolve iteration	50
	parameter $df_{percent}$ for dissolve factor	0.85

For the framework parameters, the *T* parameter determines the minimum number of stable iterations before an exchange occurs. Increasing *T* ensures that solutions produced by the base algorithms are stable and high-quality before exchanging, though this may slow overall convergence. The *M* parameter controls the iteration intervals between exchanges. By adjusting *M*, we balance the frequent exchanges for diversity with sufficient time for individual algorithm improvement. A higher *M* allows more refined solutions but reduces the frequency of cross-algorithm learning. The threshold for the minimum number of EA instances ($eas_{threshold}$) ensures that a sufficient number of evolutionary algorithms remain active, preventing the framework from converging to a single algorithm too quickly. Setting this threshold too low may reduce diversity and robustness, while higher values ensure that multiple strategies are explored.

The earliest start dissolution iteration (sd_{iter}) parameter dictates when poorly performing algorithms can start to be dissolved. It is crucial to avoid setting this parameter too low to ensure that each algorithm is given sufficient time for evaluation. If set too high, the overall convergence can be hindered. The dissolution factor ($df_{percent}$) sets a tolerance level for algorithm performance before dissolution. Fine-tuning this parameter helps maintain algorithm diversity while removing ineffective strategies. Higher $df_{percent}$ values provide greater tolerance, improving stability at the cost of slower convergence.

Additionally, the ACS algorithm includes a top-k parameter, which determines the number of solutions selected from exchanged paths. A smaller top-k value means faster convergence as fewer paths are considered, while a higher value allows for more exploration at the cost of slower convergence.

By focusing on these framework-specific parameters and the additional ACS parameter, we can effectively balance the exploration and exploitation within the MEAF framework, ensuring robust and efficient optimization. Adjustments should be based on empirical testing and performance monitoring to find the optimal settings for the specific problem at hand.

For all the 56 benchmark instances, algorithms were run 21 times to evaluate their performance more accurately, and the maximum iteration number each time was 500. Regarding the initial paths of the particle swarm optimization and the genetic algorithm, we used the nearest neighbor heuristic to generate them. The selection phase of GA uses a combination of Fitness Proportionate Selection and Truncation selection. In each iteration, we used Truncation Selection to select *ts* percent from the entire population and then we used Fitness Proportionate Selection to select parents from the filtered population to produce a new generation. For the

crossover and mutation phases of the genetic algorithm, PMX and RSM were chosen, respectively, for other parameters that occurred in the experiment, as shown in Table 2.

MEAF introduces several framework-specific parameters (T, M, sd_{iter}, df_{percent}), for which default values were used in the main experiments. A sensitivity analysis of these parameters is presented in Appendix A, as shown in Fig 5.

6.2 Result analysis

The results of the experiment comparing the elementary algorithms are shown in Table 3. This table presents the outcomes for 56 instances run on ACS, PSO, GA, and MEAVRPTW. "Avg." denotes the average cost, and "Std." stands for the standard deviation. The cost reduction is computed according to (22).

$$\text{cost reduction} = \frac{Min(ACS_{Avg.}, PSO_{Avg.}, GA_{Avg.}) - Our_{Avg.}}{Min(ACS_{Avg.}, PSO_{Avg.}, GA_{Avg.})} \tag{22}$$

As shown in Table 3, for problem sets C1 and C2, MEAVRPTW obtained the minimum mean value in all 17 instances. Except for MEAVRPTW, the ACS performed the best, winning 14 out of 17 solutions compared to GA and PSO. As for problem sets R1 and R2, MEAVRPTW also obtained the minimum mean value in 21 out of 23 instances. Except for MEAVRPTW, the ACS performed better on 17 of these instances, and the GA performed better on the remaining 6 cases. For problem sets RC1 and RC2, the performance of each algorithm is similar to the previous performance. MEAVRPTW performs better in all 16 instances. Except for MEAVRPTW, ACS performed better in all 11 instances, while GA performed better in the remaining 5 instances. Overall, the MEAVRPTW algorithm achieved 54 optimal solutions out of 56 instances.

Moreover, from the experimental run logs, we can see that the PSO algorithm dissolves 21*56=1176 runs. The experimental results can fully prove that the proposed framework can identify and filter out the bad algorithms in the runs. From the experimental data of the individually run algorithms, we can see that the algorithms perform best in sorting the results of ACS, followed by GA, and lastly comes PSO.

Some representative examples from the three data types are selected to analyze the algorithm's convergence, as shown in Fig 4. The figure shows that the stage of fast convergence of the algorithm is within 100 generations, after which the algorithm results converge slowly. For R data (i.e., map coordinates generated from random data), the MEAVRPTW algorithm has two examples where the results are not as good as the ACS and GA algorithms. The analysis is performed here, and the examples of R208 and R211 are chosen for comparison. These two examples differ only in the customer time window; 75 customers in R208 have a larger acceptable service window than those in R211, while the other 25 customers have a smaller acceptable time window than those in R211. The GA algorithm and the PSO algorithm do not consider restrictions during path generation but only schedule vehicles according to the restrictions in the final stage of execution. On the other hand, the ACS algorithm considers the limits during the initialization phase when making pheromone assignments and when calculating the next direction of the ants. When the time window is small, the advantage offered by this consideration of the ant algorithm is more apparent compared to GA and PSO, while it is not apparent when the window is larger. In addition to that. when the ACS algorithm is not as good as GA in realizing such data, it amounts to a drag on the overall performance of the VRPTW algorithm. The opposite is true for use cases R102 and R105, where the time window of use case R102 is smaller overall, and the advantage of the ACS algorithm is more evident now. Solving this problem requires considering the time limit in the generation path of GA and PSO algorithms, which can be the direction of our subsequent future research.

We can analyze the results of the four algorithms on VRPTW in terms of their search mechanisms. Initially designed for continuous optimization problems, the PSO algorithm is unsuited for VRPTW due to its reliance on particle movement and velocity updating concepts. Since VRPTW is a discrete problem, the PSO algorithm needs to be discretized, but experimental results indicate that the selected concepts of particle movement and velocity updating are ineffective in harnessing the strengths of PSO for VRPTW. This suggests that PSO requires more advanced discretization strategies to be effective for VRPTW. Existing approaches struggle to preserve feasibility and diversity under complex constraints, which may explain their relatively weaker performance compared with algorithms that are inherently more suitable for discrete combinatorial optimization. In contrast, the Ant Colony System (ACS) algorithm employs a step-by-step solution construction approach, effectively incorporating time window constraints into the route construction process. By enabling ants to communicate and share information through pheromone updates, ACS reinforces more favorable routes that adhere to time window constraints and minimize total travel in VRPTW. Regarding Genetic Algorithms (GA), the selection and mutation components are vital in driving solution evolution for VRPTW. However, due to the complexity and constraints of VRPTW, GA may yield slightly inferior results compared to ACS. Traditional GA methods often fail to explicitly consider these constraints, resulting in the generation of infeasible solutions. In contrast, ACS incorporates a pheromone-updating mechanism that considers time window constraints during

Table 3 Experiment results of MEAVRPTW compared with ACS, PSO, GA on 56 benchmark instances

	ACS		PSO		GA		MEAVRPTW		
Inst.	Avg.	Std.	Avg.	Std.	Avg.	Std.	Avg.	Std.	cost reduction(%)
C101	1044.81	51.68	1830.69	83.16	1257.19	60.71	956.51	39.13	8.45
C102	1088.0	52.63	1621.93	71.8	1215.64	41.88	1006.72	44.43	7.47
C103	1159.66	50.4	1459.52	54.65	1139.56	57.65	1030.82	39.68	9.54
C104	1098.74	36.63	1210.81	20.54	981.48	30.41	966.4	35.1	1.54
C105	988.26	52.4	1787.82	83.29	1236.19	38.14	936.14	48.37	5.27
C106	1004.86	46.3	1580.25	63.8	1175.1	56.41	926.29	29.46	7.82
C107	1016.4	46.42	1754.91	70.38	1178.57	61.56	935.36	39.56	7.97
C108	989.98	48.16	1437.32	25.04	1115.72	45.46	893.15	55.24	9.78
C109	957.94	41.98	1390.97	40.38	1069.57	41.09	891.58	47.25	6.93
C201	896.32	59.99	2314.8	110.85	1467.78	100.68	834.92	36.23	6.85
C202	1000.06	64.21	1781.67	81.97	1329.16	55.74	897.35	48.76	10.27
C203	1016.13	66.58	1522.41	81.24	1088.15	71.88	870.52	49.6	14.33
C204	988.84	63.4	1266.33	41.69	934.42	56.54	832.56	58.19	10.9
C205	852.21	71.04	1715.54	58.36	1208.98	87.79	739.78	45.3	13.19
C206	833.34	46.16	1483.95	47.52	1022.93	94.22	694.78	59.54	16.63
C207	822.57	53.24	1376.59	26.78	1012.9	76.47	693.78	51.77	15.66
C208	807.69	52.42	1399.68	21.15	937.44	81.14	687.12	27.67	14.93
R101	1816.36	45.49	2671.0	72.14	2370.34	77.14	1790.51	33.95	1.42
R102	1664.43	40.82	2368.67	61.96	2113.45	59.51	1669.92	29.58	-0.33
R103	1492.86	36.06	1905.12	22.56	1706.88	29.02	1447.03	40.34	3.07
R104	1313.88	46.63	1504.99	17.77	1353.18	23.34	1227.44	38.49	6.58
R105	1591.71	39.36	2380.23	77.15	1975.84	54.51	1570.31	30.19	1.34
R106	1503.28	33.3	2178.57	63.91	1795.68	58.3	1491.01	35.39	0.82
R107	1399.23	38.45	1804.78	34.6	1535.64	28.31	1303.45	36.16	6.85
R108	1263.79	33.91	1385.88	20.36	1194.39	28.25	1141.61	21.96	4.42
R109	1416.49	46.59	1914.22	30.54	1602.97	56.68	1362.95	28.98	3.78
R110	1360.01	29.44	1738.6	36.84	1475.49	35.97	1285.15	27.35	5.5
R111	1342.41	42.95	1731.05	6.68	1488.62	32.65	1296.34	33.89	3.43
R112	1217.0	38.07	1258.41	1.69	1163.25	9.67	1123.47	28.78	3.42
R201	1398.96	27.28	2155.02	63.47	1763.65	43.33	1377.67	45.62	1.52
R202	1342.28	31.55	1954.3	43.46	1595.91	38.31	1307.76	30.89	2.57
R203	1213.62	39.12	1628.96	37.72	1303.97	44.15	1136.36	41.07	6.37
R204	1028.37	30.6	1101.0	10.14	968.7	32.66	937.13	30.18	3.26
R205	1251.19	47.96	1767.96	55.53	1381.81	56.69	1169.45	30.7	6.53
R206	1194.0	32.47	1562.96	62.35	1238.49	44.52	1112.96	42.51	6.79
R207	1143.58	40.98	1387.3	39.57	1101.29	32.22	1072.13	42.03	2.65
R208	988.3	36.17	1019.15	1.4	863.49	13.14	867.03	13.68	-0.41
R209	1153.62	37.27	1537.71	27.67	1235.34	27.49	1072.36	66.5	7.04
R210	1227.62	40.84	1607.92	16.86	1234.44	35.81	1135.84	36.83	7.48
R211	1031.48	44.89	1076.49	4.63	981.97	14.13	928.51	32.8	5.44
RC101	1869.4	34.98	2945.6	96.86	2255.04	88.89	1850.73	41.97	1.0
RC102	1755.16	36.62	2611.94	65.14	1978.42	78.38	1696.61	45.29	3.34
RC103	1610.8	43.89	2031.84	42.73	1683.65	38.44	1539.02	63.26	4.46
RC104	1442.3	44.89	1623.7	16.04	1415.24	39.22	1317.8	39.18	6.89
RC105	1833.82	47.04	2612.54	77.68	2068.63	102.8	1766.69	53.96	3.66
RC106	1655.47	50.66	2213.29	58.8	1776.51	54.84	1571.39	54.02	5.08
RC107	1541.58	52.71	1773.51	47.78	1563.38	48.84	1447.09	40.67	6.13
RC108	1412.17	42.65	1499.19	18.81	1363.77	25.75	1292.23	28.46	5.25
RC201	1616.07	48.61	2479.11	131.61	1925.35	75.89	1540.29	50.87	4.69
RC202	1467.89	45.3	2034.31	60.48	1620.76	56.38	1365.39	74.45	6.98
RC203	1299.81	42.42	1580.98	77.14	1334.51	40.81	1233.06	54.65	5.14
RC204	1111.3	40.77	1180.24	24.01	962.91	16.31	946.22	25.27	1.73
RC205	1509.02	49.88	2216.77	96.93	1707.16	65.32	1442.59	52.92	4.4
RC206	1386.74	56.35	2268.69	98.06	1561.41	79.3	1277.91	42.96	7.85

Table 3 (continued)

	ACS		PSO		GA		MEAVRPTW		
Inst.	Avg.	Std.	Avg.	Std.	Avg.	Std.	Avg.	Std.	cost reduction(%)
RC207	1338.9	46.51	1625.7	64.17	1301.71	27.34	1215.14	61.83	6.65
RC208	1051.74	31.48	1076.75	1.56	966.62	17.52	926.72	31.31	4.13

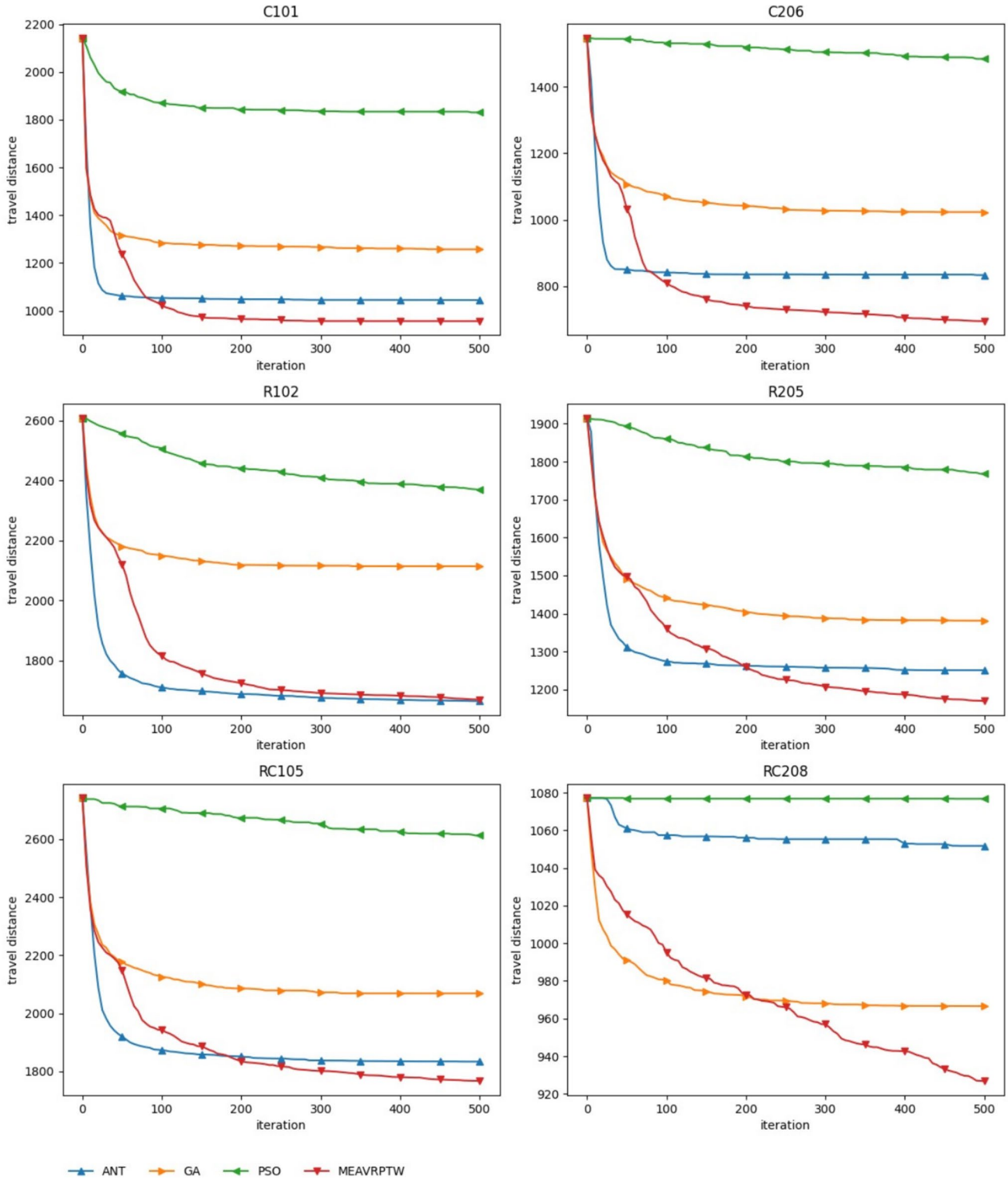

Fig. 4 Convergence graph for algorithms running with 500 iterations on 6 benchmark instances

solution construction and adaptation, making it a more suitable choice for VRPTW. Lastly, our proposed MEAVRPTW algorithm effectively synthesizes the advantages of various algorithms through communication exchange and the timely elimination of unsuitable algorithms. This approach has yielded the best results in the experimental process, showcasing its effectiveness in solving VRPTW.

Finally, we briefly analyze the execution time of the algorithms. On the test computer with the above running parameters, it takes about 21 seconds to run the ACS algorithm once, about 15 seconds to run the MEAVRPTW algorithm once, about 7 seconds to run the GA algorithm once, and about 2 seconds to run the PSO algorithm once. Since the PSO algorithm was introduced to verify that the algorithm framework can identify inferior algorithms, PSO will dissolve away in the primary stage of the run. Considering that most of the runs in the algorithm framework are ACS and GA algorithms, the running time of the hybrid algorithm is the average running time of the GA algorithm and the ACS algorithm.

The results of the experiment comparing the recently developed meta-heuristic algorithms POA, OOA, and COA are shown in Table 4. As we can see, the proposed MEAVRPTW algorithm outperforms the recently developed meta-heuristic algorithms in all 56 instances. This demonstrates the essential nature of our proposed algorithm and highlights the robustness of our framework. The consistent performance improvements indicate that MEAVRPTW provides a more effective solution, showcasing its potential as a critical advancement in the field of meta-heuristic optimization.

To evaluate the statistical significance of MEAVRPTW's performance, we first conducted a Friedman test [53] on the total travel distances obtained by all seven algorithms across 56 benchmark instances. The test yielded a statistic of 276.74 ($p < 0.001$), indicating that the differences in average rankings among the algorithms are highly significant. A larger Friedman statistic reflects greater disparity in algorithm performance across instances, with MEAVRPTW consistently achieving the lowest (best) rank. We further performed Wilcoxon signed-rank tests [54] to compare MEAVRPTW against each baseline algorithm individually. The Wilcoxon statistics ranged from 0 to 1 ($p < 0.001$), where a smaller statistic indicates that MEAVRPTW consistently outperformed the baseline across instances. A statistic of 0 means MEAVRPTW was better in every instance. These results confirm that MEAVRPTW achieves statistically significant improvements over all considered baseline algorithms in terms of total travel distance.

Finally, Table 5 in Appendix A presents the average computational time (in seconds) of ACS, PSO, GA, and MEAVRPTW over 21 runs. These results provide a quantitative comparison of the computational efficiency of the algorithms across different problem instances. MEAVRPTW achieves high-quality solutions while maintaining a reasonable computational speed, as its execution time is mainly determined by the combined computational cost of ACS, PSO, and GA components. Its computational time is generally faster than that of ACS, while being slower than PSO and GA. This intermediate runtime is expected, as MEAVRPTW integrates multiple algorithms and periodically exchanges the best-found solutions among them, balancing solution quality and computational efficiency.

To further examine scalability, we tested MEAVRPTW on selected 600-customer VRPTW instances. The single-run results are provided in Table 6 of Appendix A, which demonstrate that MEAVRPTW maintains competitive solution quality on larger-scale problems.

It should be noted that the primary goal of MEAVRPTW (MEAF) is to provide a modular framework that integrates multiple metaheuristics (ACS, PSO, GA) with minimal additional computational overhead. Therefore, the focus of this study is on demonstrating the effectiveness of this hybridization approach rather than directly comparing with all existing state-of-the-art hybrid algorithms, which often involve highly problem-specific or tailored strategies. While comprehensive benchmarking against recent hybrid methods is beyond the scope of the present work, the framework is flexible and can potentially incorporate such strategies in the future, enabling further performance improvements. This distinction has been clarified in the revised manuscript to contextualize MEAF within the broader landscape of VRPTW solution approaches.

7 Conclusions

This article is divided into two main parts. In the first part, we introduce the membrane-inspired evolutionary algorithm framework (MEAF), which combines four evolutionary rules: isolated evolutionary stability, communication out, communication in, and membrane dissolution, with information exchange operations. Inspired by membrane computation and biological evolution, MEAF aims to harness the potential of multiple evolutionary algorithms to approximate the optimal solution more effectively. By incorporating these evolutionary rules and facilitating information exchange, MEAF enables collaborative evolution and enhances search space exploration. This framework represents a novel approach that merges the principles of membrane computation and evolutionary algorithms, suggesting its potential for significant contributions to solving complex optimization problems.

In the second part, we present the MEAVRPTW algorithm, which applies the MEAF framework to address the VRPTW. The MEAVRPTW algorithm combines ACS,

Table 4 Experiment results of MEAVRPTW compared with COA, OOA, POA on 56 benchmark instances

	COA		OOA		POA		MEAVRPTW		
Inst.	Avg.	Std.	Avg.	Std.	Avg.	Std.	Avg.	Std.	cost reduction(%)
C101	1365.63	59.88	1420.04	80.95	1323.59	54.81	956.51	39.13	27.73
C102	1278.20	56.30	1266.00	50.11	1257.04	68.32	1006.72	44.43	19.91
C103	1208.10	56.71	1190.02	42.95	1150.58	57.52	1030.82	39.68	10.41
C104	1046.23	34.59	1036.36	34.34	1015.36	34.43	966.4	35.1	4.82
C105	1377.02	55.29	1373.32	77.81	1312.05	51.84	936.14	48.37	28.65
C106	1256.02	49.01	1267.98	61.28	1245.90	78.50	926.29	29.46	25.65
C107	1314.22	66.67	1336.43	71.13	1287.97	80.93	935.36	39.56	27.38
C108	1161.77	38.13	1191.06	42.98	1111.08	33.40	893.15	55.24	19.61
C109	1111.43	33.97	1125.53	44.50	1079.06	28.47	891.58	47.25	17.37
C201	1615.20	94.68	1656.88	128.39	1592.26	105.66	834.92	36.23	47.56
C202	1388.27	54.49	1389.58	79.91	1375.52	67.60	897.35	48.76	34.76
C203	1175.42	82.33	1161.70	67.81	1175.38	56.62	870.52	49.6	25.06
C204	1015.50	50.53	985.47	56.89	956.75	52.42	832.56	58.19	12.98
C205	1298.17	75.99	1236.41	65.58	1185.46	83.97	739.78	45.3	37.60
C206	1160.68	62.00	1036.40	74.41	1001.75	64.43	694.78	59.54	30.64
C207	1087.81	70.79	1016.78	78.72	972.65	75.48	693.78	51.77	28.67
C208	1005.00	70.16	928.68	59.80	922.33	60.02	687.12	27.67	25.50
R101	2433.84	70.84	2430.80	68.11	2368.28	77.63	1790.51	33.95	24.40
R102	2180.22	41.19	2163.06	46.59	2158.52	66.18	1669.92	29.58	22.64
R103	1758.59	27.74	1739.67	30.61	1722.81	31.57	1447.03	40.34	16.01
R104	1403.79	27.12	1376.20	22.70	1359.27	22.40	1227.44	38.49	9.70
R105	2042.89	59.76	2016.20	58.72	2000.91	49.13	1570.31	30.19	21.52
R106	1808.21	60.01	1859.74	65.30	1808.99	42.13	1491.01	35.39	17.54
R107	1579.03	41.19	1530.30	40.82	1531.34	47.47	1303.45	36.16	14.82
R108	1231.87	30.00	1207.98	30.77	1201.30	26.91	1141.61	21.96	4.97
R109	1678.24	34.48	1624.80	49.88	1612.65	40.60	1362.95	28.98	15.48
R110	1512.33	26.53	1472.87	35.24	1475.74	38.43	1285.15	27.35	12.75
R111	1551.05	31.42	1500.10	34.42	1497.85	27.45	1296.34	33.89	13.45
R112	1185.97	12.08	1163.21	11.61	1160.50	13.23	1123.47	28.78	3.19
R201	1838.41	37.71	1797.00	34.03	1804.85	46.95	1377.67	45.62	23.34
R202	1657.68	36.01	1638.28	59.99	1618.98	54.81	1307.76	30.89	19.22
R203	1430.40	38.11	1353.23	47.97	1335.91	42.12	1136.36	41.07	14.94
R204	1006.21	19.86	980.79	22.12	985.04	27.01	937.13	30.18	4.45
R205	1508.53	41.02	1428.69	58.33	1463.50	55.74	1169.45	30.7	18.15
R206	1347.22	44.41	1287.16	39.16	1300.97	56.11	1112.96	42.51	13.53
R207	1147.69	30.54	1124.49	39.10	1123.68	44.51	1072.13	42.03	4.59
R208	891.91	21.53	879.05	12.85	880.98	20.38	867.03	13.68	1.37
R209	1306.91	16.43	1275.12	27.06	1260.37	40.33	1072.36	66.5	14.92
R210	1327.28	41.77	1277.33	47.32	1279.37	35.62	1135.84	36.83	11.08
R211	1017.33	19.42	1016.44	12.47	969.08	25.45	928.51	32.8	4.19
RC101	2336.26	90.12	2347.04	56.43	2311.75	97.14	1850.73	41.97	19.94
RC102	2093.37	65.39	2007.98	71.05	2017.88	73.81	1696.61	45.29	15.51
RC103	1768.69	46.87	1701.22	38.84	1687.61	48.31	1539.02	63.26	8.80
RC104	1461.76	36.16	1424.10	42.01	1403.29	39.82	1317.8	39.18	6.09
RC105	2143.06	75.59	2054.30	64.71	2075.13	68.55	1766.69	53.96	14.00
RC106	1831.98	62.67	1787.27	47.47	1792.82	49.78	1571.39	54.02	12.08
RC107	1567.88	56.39	1580.36	56.29	1561.05	57.65	1447.09	40.67	7.30
RC108	1393.78	23.04	1382.39	23.87	1375.37	27.11	1292.23	28.46	6.04
RC201	2010.30	82.07	1934.75	91.01	1948.36	92.49	1540.29	50.87	20.39
RC202	1670.80	58.70	1627.93	48.61	1594.36	44.99	1365.39	74.45	14.36
RC203	1389.10	37.40	1351.62	46.63	1348.91	66.95	1233.06	54.65	8.59
RC204	1013.84	20.23	976.79	18.83	985.63	40.53	946.22	25.27	3.13
RC205	1856.48	69.21	1791.09	74.29	1750.84	86.22	1442.59	52.92	17.61
RC206	1718.23	72.28	1630.60	67.64	1599.59	80.09	1277.91	42.96	20.11

Table 4 (Continued)

	COA		OOA		POA		MEAVRPTW		
Inst.	Avg.	Std.	Avg.	Std.	Avg.	Std.	Avg.	Std.	cost reduction(%)
RC207	1374.17	50.50	1306.50	34.41	1277.88	35.96	1215.14	61.83	4.91
RC208	996.71	14.32	986.21	23.33	968.13	24.58	926.72	31.31	4.28

GA, and PSO to form the elementary algorithm within the MEAF's framework. We conducted our experiments using 56 instances from Solomon's VRPTW benchmark to evaluate its performance. Comparing the results of the MEAVRPTW algorithm with the elementary algorithms GA, ACS, and PSO, MEAVRPTW achieved superior performance on average in 54 out of the 56 instances. Notably, our analysis of the experimental programs log revealed that all PSO running units were eliminated during the early stages of evolution. This proves that the MEAF algorithm framework effectively identifies and discards algorithms unsuitable for solving specific problem types. Furthermore, by observing the experimental result data and convergence curves, we found that the MEAVRPTW algorithm successfully combines the evolutionary advantages of multiple algorithms and efficiently explores the search space through inter-algorithm communication, even when the basic algorithm reaches convergence. The exceptional performance demonstrated by the MEAVRPTW algorithm serves as a verification of the MEAF's framework efficacy in solving real-world problems.

Appendix A: Additional results

Table 5 Average computational time (in seconds) of ACS, PSO, GA, and MEAVRPTW after 21 runs

inst.	ACS	PSO	GA	MEAVRPTW
C101	12.81	0.62	2.57	8.29
C102	14.00	0.67	2.43	7.67
C103	13.24	0.67	2.57	7.71
C104	13.67	0.71	2.86	5.43
C105	13.90	0.62	2.67	7.90
C106	13.86	0.67	2.52	7.90
C107	13.86	0.62	2.48	7.90
C108	13.71	0.71	2.62	7.76
C109	13.62	0.76	2.62	7.81
C201	12.10	0.62	2.43	6.76
C202	12.24	0.62	2.38	6.71
C203	12.19	0.67	2.38	6.57
C204	12.10	0.76	2.29	6.57
C205	10.95	0.76	2.48	6.62
C206	11.86	0.76	2.38	6.52
C207	11.10	0.76	2.29	6.48
C208	11.86	0.76	2.29	6.48
R101	17.38	0.76	2.76	9.38
R102	15.10	0.81	2.67	8.76
R103	15.43	0.81	2.81	7.33

Table 5 (Continued)

inst.	ACS	PSO	GA	MEAVRPTW
R104	14.29	0.76	3.05	6.14
R105	16.24	0.86	2.71	8.86
R106	15.48	0.86	2.62	8.33
R107	14.76	0.81	2.86	7.86
R108	14.10	0.76	2.57	5.81
R109	15.05	0.86	2.48	8.10
R110	14.29	0.81	2.48	7.67
R111	14.57	0.81	2.48	7.14
R112	12.76	0.76	2.48	5.57
R201	15.24	0.81	2.48	8.33
R202	13.95	0.81	2.38	7.71
R203	13.38	0.76	2.33	7.86
R204	12.43	0.71	2.19	5.62
R205	14.14	0.81	2.29	7.71
R206	13.00	0.76	2.24	7.19
R207	12.81	0.71	2.24	7.19
R208	12.14	0.71	2.19	5.29
R209	13.00	0.76	2.24	7.05
R210	13.48	0.76	2.29	7.48
R211	12.57	0.71	2.24	5.29
RC101	16.29	0.86	2.67	9.00
RC102	15.71	0.86	2.90	8.67
RC103	13.95	0.81	2.48	7.95
RC104	13.76	0.81	2.81	6.19
RC105	15.86	0.86	2.86	8.81
RC106	14.86	0.81	2.62	8.00
RC107	14.33	0.81	2.71	7.05
RC108	14.14	0.76	2.57	5.86
RC201	14.90	0.76	2.57	8.14
RC202	14.38	0.76	2.43	7.90
RC203	13.24	0.71	2.24	7.95
RC204	11.33	0.71	2.24	6.00
RC205	13.48	0.76	2.48	7.90
RC206	13.71	0.76	2.52	7.19
RC207	13.24	0.71	2.38	6.95
RC208	12.43	0.71	2.24	5.43

Table 6 Single-run results on large-scale (600-customer) VRPTW instances

Instance	MEAVRPTW	ACS	PSO	GA
C101	27151.19	28853.21	65006.46	40857.38
C201	26720.28	27913.78	60873.56	38271.23
R101	47724.40	48230.03	68252.41	53730.20
R201	39999.15	**39756.44**	63796.92	48508.68
RC101	34491.03	36401.21	52332.76	41963.90
RC201	30587.96	30770.70	48519.25	36989.36

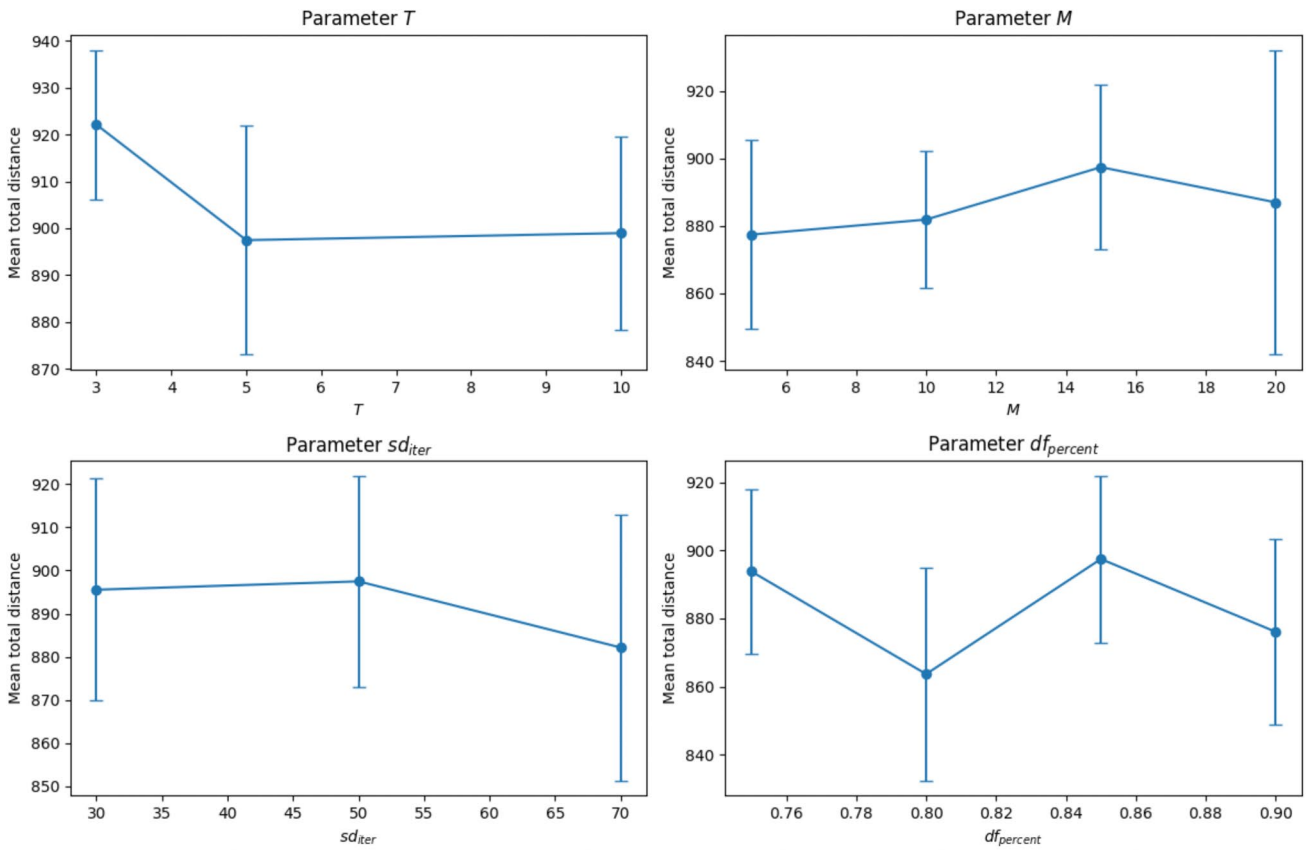

Fig. 5 Sensitivity analysis of four key MEAF parameters on the Solomon C101 instance. The y-axis shows the mean total distance over 5 independent runs for each parameter configuration. Parameters analyzed are T (minimum iterations before exchange), M (exchange interval), sd_{iter} (earliest iteration to start dissolution), and df_{percent} (dissolve factor). Error bars indicate one standard deviation, illustrating algorithm stability. The results show that MEAF is generally robust across the tested parameter ranges

Based on the sensitivity analysis, MEAF is robust to variations in the parameters. The recommended ranges for practical usage are $T = 3$–10, $M = 5$–20, $sd_{\text{iter}} = 30$–70, and $df_{\text{percent}} = 0.75$–0.90. These ranges provide guidance for parameter tuning while maintaining algorithmic stability and performance.

Acknowledgements This scientific result is part of the CLARA project that has received funding from the European Union's HORIZON EUROPE research and innovation programme under Grant Agreement No 101136607. The authors gratefully acknowledge financial support ROBOPROX of No. CZ.02.01.01/00/22 008/0004590 by Ministry of Education, Youth, and Sports; and European Union under the REFRESH — Research Excellence For Region Sustainability and High-tech Industries project number CZ.10.03.01/00/22_/0000048 via the Operational Programme Just Transition;

Funding Open access publishing supported by the institutions participating in the CzechELib Transformative Agreement.

Data Availability The dataset used for this work is available at the following link: https://www.sintef.no/projectweb/top/vrptw/100-customers/.

Declarations

Conflict of Interest The authors have no conflicts of interest to disclose that relate to the subject matter of this article.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Dantzig GB, Ramser JH (1959) The truck dispatching problem. Manage Sci 6(1):80–91

2. Alipour MM, Emami H, Abdolhosseinzadeh M (2022) A mas approach for vehicle routing problem. Neural Comput Appl 34(6):4387–4411
3. Lehmann J, Winkenbach M (2024) A matheuristic for the two-echelon multi-trip vehicle routing problem with mixed pickup and delivery demand and time windows. Transportation Research Part C Emerging Technologies 160:104522
4. RamachandranPillai R, Arock M (2021) Spiking neural firefly optimization scheme for the capacitated dynamic vehicle routing problem with time windows. Neural Comput Appl 33(1):409–432
5. Yue B, Yang J, Ma J, Shi J, Shangguan L (2024) An improved sequential insertion algorithm and tabu search to vehicle routing problem with time windows. RAIRO-Operations Research 58(2):1979–1999
6. Zhang J, Li Y, Lu Z (2024) Multi-period vehicle routing problem with time windows for drug distribution in the epidemic situation. Trans Res Part C Emerg Technol 160:104484
7. El-Sherbeny NA (2010) Vehicle routing with time windows: An overview of exact, heuristic and metaheuristic methods. Journal of King Saud University-Science 22(3):123–131
8. Mitchell M (1998) An Introduction to Genetic Algorithms. MIT press, Cambridge, MA
9. Van Laarhoven PJ, Aarts EH, Laarhoven PJ, Aarts EH (1987) Simulated Annealing. Springer, Dordrecht
10. Glover, F., Laguna, M.: Tabu Search, pp. 2093–2229. Springer, Boston, MA (1998)
11. Kennedy, J., Eberhart, R.: Particle swarm optimization. In: Proceedings of ICNN'95-international conference on neural networks, IEEE, vol. 4, pp. 1942–1948 (1995)
12. Dorigo M, Birattari M, Stutzle T (2006) Ant colony optimization. IEEE Comput Intell Mag 1(4):28–39
13. Shehadeh HA (2023) Chernobyl disaster optimizer (cdo): a novel meta-heuristic method for global optimization. Neural Comput Appl 35(15):10733–10749
14. Bouaouda, A., Hashim, F.A., Sayouti, Y., Hussien, A.G.: Pied kingfisher optimizer: a new bio-inspired algorithm for solving numerical optimization and industrial engineering problems. Neural Comput Appl pp 1–59 (2024)
15. Dehghani M, Montazeri Z, Trojovská E, Trojovskỳ P (2023) Coati optimization algorithm: A new bio-inspired metaheuristic algorithm for solving optimization problems. Knowl-Based Syst 259:110011
16. Alsalibi B, Mirjalili S, Abualigah L, Yahya RI, Gandomi AH (2022) A comprehensive survey on the recent variants and applications of membrane-inspired evolutionary algorithms. Arch Comput Method Eng pp 1–17
17. Zhang G, Gheorghe M, Pan L, Pérez-Jiménez MJ (2014) Evolutionary membrane computing: a comprehensive survey and new results. Inf Sci 279:528–551
18. Desrosiers J, Soumis F, Desrochers M (1984) Routing with time windows by column generation. Networks 14(4):545–565
19. Desrochers M, Desrosiers J, Solomon M (1992) A new optimization algorithm for the vehicle routing problem with time windows. Oper Res 40(2):342–354
20. Fisher ML, Jörnsten KO, Madsen OB (1997) Vehicle routing with time windows: Two optimization algorithms. Oper Res 45(3):488–492
21. Kohl N, Desrosiers J, Madsen OB, Solomon MM, Soumis F (1999) 2-path cuts for the vehicle routing problem with time windows. Transp Sci 33(1):101–116
22. Balas E, Guignard M (1979) Report of the session on: Branch and bound/implicit enumeration. In: Annals of discrete mathematics vol 5, pp 185–191. Elsevier, Amsterdam
23. Baker EK, Schaffer JR (1986) Solution improvement heuristics for the vehicle routing and scheduling problem with time window constraints. Am J Math Manag Sci 6(3–4):261–300
24. Van Landeghem H (1988) A bi-criteria heuristic for the vehicle routing problem with time windows. Eur J Oper Res 36(2):217–226
25. Potvin J-Y, Rousseau J-M (1993) A parallel route building algorithm for the vehicle routing and scheduling problem with time windows. Eur J Oper Res 66(3):331–340
26. Potvin J-Y, Rousseau J-M (1995) An exchange heuristic for routeing problems with time windows. J Oper Res Soc 46(12):1433–1446
27. Dorigo M, Gambardella LM (1997) Ant colony system: a cooperative learning approach to the traveling salesman problem. IEEE Trans Evol Comput 1(1):53–66
28. Mirjalili S, Lewis A (2016) The whale optimization algorithm. Adv Eng Softw 95:51–67
29. Wang Y, Chen X, Shuang Z, Zhan Y, Chen K, Xu C (2024) Self-competition particle swarm optimization algorithm for the vehicle routing problem with time window. IEEE Access
30. Maroof A, Ayvaz B, Naeem K (2024) Logistics optimization using hybrid genetic algorithm (hga): a solution to the vehicle routing problem with time windows (vrptw). IEEE Access
31. Widians JA, Wardoyo R, Hartati S (2024) A hybrid ant colony and grey wolf optimization algorithm for exploitation-exploration balance. Emerg Sci J 8(4):1642–1654
32. Farda I, Thammano A (2024) An adaptive differential evolution with multiple crossover strategies for optimization problems. HighTech Innov J 5(2):231–58
33. Arif MS, Abodayeh K, Nawaz Y (2024) A third-order two-stage numerical scheme and neural network simulations for seir epidemic model: A numerical study. Emerg Sci J 8(1)
34. Dehghani M, Trojovskỳ P (2023) Osprey optimization algorithm: A new bio-inspired metaheuristic algorithm for solving engineering optimization problems. Front Mech Eng 8:1126450
35. Trojovskỳ P, Dehghani M (2022) Pelican optimization algorithm: A novel nature-inspired algorithm for engineering applications. Sensors 22(3):855
36. Tan KC, Lee LH, Zhu Q, Ou K (2001) Heuristic methods for vehicle routing problem with time windows. Artif Intell Eng 15(3):281–295
37. Păun G (2000) Computing with membranes. J Comput Syst Sci 61(1):108–143
38. Ciobanu G, Păun G, Pérez-Jiménez MJ (2006) Applications of Membrane Computing, vol 17. Springer, Berlin
39. Colorni A, Dorigo M, Maniezzo V et al (1991) Distributed optimization by ant colonies. In: Proceedings of the first european conference on artificial life, Paris, France, vol 142, pp 134–142
40. Schoonderwoerd R, Holland OE, Bruten JL, Rothkrantz LJ (1997) Ant-based load balancing in telecommunications networks. Adapt Behav 5(2):169–207
41. Goldberg DE, Deb K (1991) A comparative analysis of selection schemes used in genetic algorithms. In: Foundations of genetic algorithms, Elsevier, Amsterdam, vol 1, pp 69–93
42. Umbarkar AJ, Sheth PD (2015) Crossover operators in genetic algorithms: a review. ICTACT J Soft Comput 6(1)
43. Goldberg DE, Lingle R (2014) Alleles, loci, and the traveling salesman problem. In: Proceedings of the first international conference on genetic algorithms and their applications, Psychology Press, pp 154–159
44. Davis L et al (1985) Applying adaptive algorithms to epistatic domains. In: IJCAI, Citeseer, 85:162–164
45. Abdoun O, Abouchabaka J, Tajani C (2012) Analyzing the performance of mutation operators to solve the travelling salesman problem. arXiv preprint arXiv:1203.3099
46. Onwubolu GC, Babu B, Clerc M (2004) Discrete particle swarm optimization, illustrated by the traveling salesman problem. New Optim Techn Eng pp 219–239

47. Hendtlass T (2003) Preserving diversity in particle swarm optimisation. In: Developments in applied artificial intelligence: 16th international conference on industrial and engineering applications of artificial intelligence and expert systems, IEA/AIE 2003 Loughborough, UK, June 23–26, 2003 Proceedings, Springer, 16:31–40
48. Wang K-P, Huang L, Zhou C-G, Pang W (2003) Particle swarm optimization for traveling salesman problem. In: Proceedings of the 2003 international conference on machine learning and cybernetics (IEEE Cat. No. 03ex693), IEEE, 3:1583–1585
49. Tsai C-W, Tseng S-P, Chiang M-C, Yang C-S, Hong T-P (2014) A high-performance genetic algorithm: using traveling salesman problem as a case. Sci World J 2014
50. Sohail MS, Saeed MOB, Rizvi SZ, Shoaib M, Sheikh AUH (2014) Low-complexity particle swarm optimization for time-critical applications. arXiv preprint arXiv:1401.0546
51. Li M (2015) Efficiency improvement of ant colony optimization in solving the moderate ltsp. J Syst Eng Electron 26(6):1300–1308
52. Solomon MM (1987) Algorithms for the vehicle routing and scheduling problems with time window constraints. Oper Res 35(2):254–265
53. Friedman M (1937) The use of ranks to avoid the assumption of normality implicit in the analysis of variance. J Am Stat Assoc 32(200):675–701
54. Wilcoxon F (1945) Individual comparisons by ranking methods. Biometrics bulletin 1(6):80–83

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.